🏗️ Arquitectura General del Sistema

🎯 Estilo Arquitectónico

Layered Architecture + Modular Analytics Integration

No microservicios.
No monolito desordenado.
Es un **modular monolith bien estructurado** con integración analítica orquestada.

📐 Vista de Alto Nivel (Container-Level)

```
┌──────────────────────────┐
│      Frontend            │
│ Vue 3 + Vite + TS        │
│ (SPA)                    │
└──────────────┬───────────┘
               │ HTTPS (JWT)
┌──────────────▼───────────┐
│      Backend API         │
│ Node.js + Express + TS   │
│ Arquitectura en capas    │
└──────┬──────────┬────────┘
       │          │
       │          │
       │          ▼
       │   ┌──────────────┐
       │   │ Analytics Job │
       │   │ node-cron     │
       │   └──────┬───────┘
       │          │ spawn()
       ▼          ▼
┌──────────────┐ ┌──────────────
```

```
│ PostgreSQL  │  │ Python Module  │
│ Prisma ORM  │  │ pandas + ML    │
└──────────────┘  └────────────────┘
```

🔍 Componentes y Responsabilidades

1️⃣ Frontend (Vue 3)

Responsabilidades:

- Presentación
- Manejo de estado
- Consumo de API
- Renderizado de dashboards

NO:

- Lógica de negocio
- Cálculos analíticos

2️⃣ Backend (Node + Express)

Es el **núcleo del sistema**.

División interna:

Controller Layer

↓

Service Layer

↓

Repository Layer

↓

Prisma ORM

🔹 **Controller**

- Maneja request/response
- Valida input
- Llama servicios

🔹 **Service**

- Reglas de negocio
- Cálculo IGE
- Autorización por rol
- Orquestación analytics

🔹 **Repository**

- Acceso a datos
- Abstracción de Prisma

Esto demuestra separación de responsabilidades.

3️⃣ PostgreSQL

Es:

- Sistema transaccional
- Almacén de métricas
- Registro de versiones del modelo

Debe incluir:

- Índices compuestos
- Constraints únicos
- Foreign keys
- Enum roles

4️⃣ Módulo Analítico (Python)

Es un módulo desacoplado que:

- Extrae datos
- Limpia texto
- Vectoriza
- Entrena modelo
- Evalúa
- Guarda resultados

No expone API.
Es ejecutado por Node.

🔄 Flujo de Orquestación Analítica

1. node-cron ejecuta job programado
2. Backend llama a:
3. spawn('python', ['pipeline.py'])
4. Python:
    - Consulta DB
    - Ejecuta ETL
    - Entrena modelo
    - Calcula métricas
    - Inserta resultados
5. Devuelve JSON con:
    - accuracy
    - precision
    - recall
    - f1
6. Node registra model_version

🧠 Diagrama de Secuencia (Ejecución Analytics)

node-cron

↓

AnalyticsService

↓

Python Process

↓

PostgreSQL

↓

model_versions update

🧩 Principios Arquitectónicos Aplicados

✓ Separation of Concerns

Frontend ≠ Backend ≠ Analytics

✓ Single Responsibility

Cada módulo tiene una función clara.

✓ Inversion of Control

Services dependen de interfaces (repos).

✓ Reproducibilidad

Pipeline ejecutable independientemente.

✓ Escalabilidad futura

Podría convertirse en microservicio.

🐳 Arquitectura de Contenedores (Docker)

services:

frontend

backend

postgres

analytics

Todos en la misma red Docker interna.

🔒 Seguridad

- JWT firmado
- Middleware de roles
- Sanitización de input
- No exponer ejecución Python vía API pública

⚠️ Decisiones Técnicas Justificadas

Decisión	Justificación
Monolito modular	Menor complejidad para 8 semanas
Python separado	Aislar ML del runtime Node
Prisma	Seguridad tipada + productividad
Cron interno	Control total del pipeline
Versionado modelo	Trazabilidad y reproducibilidad